



南京大學

NANJING UNIVERSITY

课程作业报告

《面向 AI 生成代码的人工审核与智能支持》实验报告

学生姓名：顾奕博、周楚然、尚昊毅、姚凯方

学生学号：231250109、231250111、231250110、231250097

2026 年 6 月

1. 案例研究设计

1.1 研究背景与动机

1.1.1 生成式人工智能对现代代码评审的范式重塑

以大语言模型（LLM）为核心的生成式人工智能技术正在重塑软件工程全生命周期的实践，特别是诸如 GitHub Copilot、ChatGPT 和 DeepSeek-Coder 等代码生成工具，已经广泛渗透到日常的软件开发流程中。

这一范式转变虽然显著降低了代码编写的门槛并极大地释放了生产力，但同时也让传统的软件质量保障体系，尤其是代码评审（Code Review）环节，面临着前所未有的挑战。

在软件工程 3.0 时代背景下，代码评审正经历深刻的范式转变，主要体现在以下三个核心痛点：

1. 代码生成规模呈指数级激增

AI 能够在数秒内快速生成大批量代码片段，导致人工审核吞吐量面临严重瓶颈，传统逐行审查模式难以为继。

2. 缺陷模式发生隐蔽性异变

AI 生成代码通常语法正确、结构规范，但可能隐藏逻辑幻觉（Hallucination）、上下文不一致以及安全越权等深层缺陷。

3. 人机协同与信任体系失衡

评审人员容易在过度信赖与完全不信任之间摇摆，亟需建立“人类主导、AI 增强”的协同评审范式。

1.1.2 智能评审支持技术在软件工程中的落地缺口

为了缓解人工审核压力，学术界与工业界提出了大量基于大模型、图神经网络以及机器学习的智能支持技术。

然而其实际落地仍存在以下问题：

1. 场景适配性不足；
2. 人机协同路径不清晰；
3. 多维效果评估不充分。

基于此，本研究通过开展准实验案例研究，系统评估多类智能代码评审支持技术在 AI 生成代码场景下的实际表现与赋能机制。

1.2 研究问题设计

本研究围绕以下四个研究问题展开：

- **RQ1：** AI 生成代码是否显著增加人工评审负担？
- **RQ2：** 不同智能评审支持工具在 AI 生成代码场景中的实际效果如何？
- **RQ3：** 不同技术路径之间是否存在显著差异？
- **RQ4：** 代码复杂度、长度等因素是否会调节智能评审工具的辅助效果？

1.3 案例研究设计六要素

表 1: 案例研究设计六要素	
要素	内容
案例对象	AI 生成代码的人机协作代码评审流程
案例目的	分析 AI 代码评审负担并评估智能支持效果
案例数量	3 个典型项目案例
分析单元	单个 AI 生成代码片段及对应评审行为
数据来源	行为日志、评审质量数据、问卷数据
分析方法	ANOVA、Kruskal-Wallis、回归分析、扎根理论

2. 实验设计

2.1 技术路径梯度设计

为了系统解构不同智能化程度的自动化技术对人工审核过程的赋能机制，本研究构建了从传统规则驱动方法到生成式人工智能评审智能体的技术演进梯度：

规则驱动静态分析 → 混合机器学习评审模型 → LLM 代码评审智能体

2.1.1 Method-A：规则驱动静态分析

Method-A 采用工业级静态分析平台 SonarQube 9.9 LTS 作为传统自动化评审基线。

实验中启用了针对 Java 与 Python 语言的核心规则集，共计约 300 条规则。系统通过抽象语法树（AST）分析、控制流分析以及安全规则匹配，对代码进行自动检测。

在人机协同流程中，Method-A 采用前置式过滤机制，在检测到潜在违规代码时自动进行高亮标记，并提供标准化问题描述及修复建议。

其主要特点如下：

- 基于预定义规则匹配；

- 能够快速发现语法和规范问题；
- 可解释性较强；
- 对复杂语义缺陷识别能力有限。

2.1.2 Method-B: 混合机器学习评审模型

Method-B 采用数据驱动的机器学习评审框架。

首先提取代码结构与语义相关的 28 项静态度量指标，包括：

- 圈复杂度 (Cyclomatic Complexity)
- 最大嵌套深度
- 代码行数
- 注释比例
- 函数调用数量
- 逻辑耦合度

其中圈复杂度通过 Radon 工具计算获得。

随后结合开源项目历史缺陷数据，训练 XGBoost 二分类模型，对代码片段出现隐藏缺陷的概率进行预测。

在人机协同过程中，该方法以风险优先级排序 (Prioritization) 的形式辅助审阅者：

- 预测缺陷概率；
- 按风险等级排序；
- 引导审阅者优先关注高风险区域。

2.1.3 Method-C: LLM 代码评审智能体

Method-C 采用基于 CodeLlama-13B-Instruct 的生成式代码评审智能体。

模型采用 LoRA (Low-Rank Adaptation) 技术进行参数高效微调，其配置如下：

- LoRA Rank: $r = 8$
- Scaling Factor: $\alpha = 16$
- Dropout: 0.1
- Epoch: 3

训练数据由人工标注的：

代码 → 缺陷 → 重构建议

三元组构成。

此外，引入检索增强生成（RAG）机制，通过动态检索历史最佳修复案例提升评审质量。

推理阶段采用 vLLM 推理框架实现高吞吐部署。

在人机协同场景下，Method-C 能够：

- 自动解释缺陷原因；
- 生成自然语言评审意见；
- 给出修复建议；
- 提供可直接应用的重构代码片段。

2.2 评价指标体系

为了从多个维度评估智能支持技术的实际效果，本研究构建了质量、效率和主观体验三类指标体系。

2.2.1 质量维度指标

缺陷检出率（Recall）

$$Recall = \frac{TP}{TP + FN} \quad (1)$$

其中：

- TP：正确识别的真实缺陷数量
- FN：漏检缺陷数量

意见准确率（Precision）

$$Precision = \frac{TP}{TP + FP} \quad (2)$$

其中：

- FP：误报缺陷数量

综合质量指标（F1-Score）

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (3)$$

2.2.2 效率维度指标

平均审核时长

$$Time = \frac{\sum(t_{submit} - t_{load})}{N}$$

(4)

其中：

- t_{load} ：代码加载时间
- t_{submit} ：提交时间

决策定位延迟

$$Latency = t_{first_interact} - t_{load}$$

(5)

2.2.3 主观体验指标

NASA-TLX 认知负荷 采用 NASA-TLX 六维量表：

1. Mental Demand
2. Physical Demand
3. Temporal Demand
4. Performance
5. Effort
6. Frustration

最终计算加权总分（0–100）。

工具信任度 采用 5 点 Likert 量表设计的三题信任度问卷。

内部一致性检验结果：

$$Cronbach's\ \alpha = 0.86$$

(6)

2.3 评价指标汇总表

表 2: 实验评价指标体系

指标类别	指标名称	统计定义	理想方向
质量维度	Recall	$\frac{TP}{TP+FN}$	↑
质量维度	Precision	$\frac{TP}{TP+FP}$	↑

指标类别	指标名称	统计定义	理想方向
质量维度	F1-Score	综合质量指标	↑
质量维度	可理解性	Likert 五点评分	↑
效率维度	审核时长	平均任务完成时间	↓
效率维度	决策延迟	首次交互时间差	↓
主观体验	NASA-TLX	认知负荷评分	↓
主观体验	工具信任度	Likert 五点评分	↑

3. 案例研究实施过程

3.1 代码评审数据集构建

为了真实模拟 AI 生成代码场景，本研究采用双源混合策略构建实验数据集。
最终形成包含 212 个有效分析单元的数据集，覆盖 Java 与 Python 两种主流编程语言。

3.1.1 真实项目数据采集

研究团队从 GitHub 开源社区筛选 10 个活跃项目。
筛选标准包括：

- 存在 Copilot 辅助开发记录；
- 存在自动化 Agent 提交历史；
- 具有持续维护活动。

共抽取 100 个存在重构或缺陷修复记录的 Pull Request 样本。

3.1.2 AI 生成代码构造

基于 CodeLlama-13B 与 GitHub Copilot 生成代码。
设计 10 类典型工业开发任务：

- 高并发任务调度
- 文件流处理
- 用户鉴权
- 网络通信
- 线程同步
- 数据缓存

- API 访问控制
- 消息队列
- 异常处理
- 数据持久化

随后实施 Defect Injection 过程。

3.1.3 缺陷注人类别

共构造五类典型高危缺陷：

1. 边界条件溢出；
2. 资源泄漏；
3. 并发竞态与死锁；
4. 权限验证缺失；
5. 不安全反序列化。

3.1.4 数据集分布

语言分布：

- Java： 50%
- Python： 50%

复杂度分布：

- 低复杂度（1-10）： 40%
- 中复杂度（11-30）： 40%
- 高复杂度（>30）： 20%

代码长度分布：

- <50 行： 40%
- 50-150 行： 40%
- >150 行： 20%

3.2 专家标注基准校准与一致性评估

为了保证实验评估过程中缺陷判定标准的准确性与一致性，研究团队邀请了两名具有五年以上互联网企业开发经验的资深 Java/Python 架构师担任领域专家，共同构建实验所需的缺陷金标准（Ground Truth）。

3.2.1 第一轮独立标注

两位专家在完全独立且互不可见的条件下，对全部 212 个代码片段进行逐项标注，具体包括：

- 缺陷所在代码行号；
- 缺陷所属类别；
- 缺陷严重程度等级（1-5 级）；
- 缺陷产生原因说明。

缺陷类别依据国际通用软件漏洞分类标准 Common Weakness Enumeration（CWE）进行统一归类。

第一轮标注结束后，研究团队采用 Cohen's κ 系数评估标注一致性：

$$\kappa = 0.71 \quad (7)$$

结果表明专家之间存在一定程度的判断分歧。

3.2.2 仲裁与标准统一

针对存在争议的标注结果，研究团队组织专题仲裁会议。

在第三名高级研究专家主持下，两位标注专家对所有分歧案例逐一讨论，并统一判定标准。

最终形成如下核心准则：

若代码问题仅影响可维护性而不会导致程序在任何运行环境下产生异常行为，则归类为代码异味（Code Smell）；若在特定输入、异常流程或并发场景下可能引发功能异常或系统崩溃，则归类为功能缺陷（Logic Bug）。

同时构建包含 50 个典型争议案例的标准判定库，供后续标注参考。

3.2.3 第二轮校准标注

完成规则统一后，两位专家再次独立开展第二轮盲标。

最终一致性系数提升至：

$\kappa = 0.84$

(8)

达到实证软件工程研究推荐的高一致性标准。
所有剩余争议样本均由第三位专家最终裁决。

3.3 参与者招募与培训

3.3.1 参与者筛选

研究团队在南京大学软件学院公开招募实验参与者。
为确保评审质量，设定如下准入条件：

- 1. 完成高级程序设计课程；
- 2. 至少具备三次项目开发经历；
- 3. 熟悉 Java 或 Python 开发；
- 4. 具备代码评审基础经验。

所有候选者均需完成预实验测试。
测试内容包括：

- Java 代码缺陷识别；
- Python 代码缺陷识别；
- 设计模式错误分析；
- 并发问题定位。

测试共 10 题，限时 20 分钟。
最终保留正确率不低于 80
共有 24 名参与者通过筛选并进入正式实验。

3.3.2 参与者基本情况

表 3: 参与者基本信息统计

指标	数值
参与者总数	24
平均年龄	23.6 岁
本科生比例	66.7%
研究生比例	33.3%
平均项目经验	3.8 个项目
平均代码评审经验	2.4 年

3.3.3 统一培训

正式实验开始前，所有参与者接受 30 分钟统一培训。

培训内容包括：

1. 缺陷分类标准；
2. 在线评审平台操作流程；
3. 三类智能工具功能介绍；
4. 示例案例讲解。

培训结束后，参与者需完成两道练习题。

确认能够熟练操作系统后方可进入正式实验。

3.4 准实验流程设计

3.4.1 实验环境控制

实验统一安排 in 软件工程实验室完成。

所有实验终端配置完全一致：

- CPU：Intel Core i7
- 内存：16GB
- 显示器：24 英寸
- 网络：千兆局域网

通过统一硬件环境降低外部干扰因素。

3.4.2 分组策略

采用分层随机化（Stratified Randomization）方法。

根据预实验成绩排序后平均分配至三个实验组：

表 4: 实验组设置

实验组	人数
Method-A	8
Method-B	8
Method-C	8

组间能力差异经统计检验不存在显著性差异。

3.4.3 拉丁方控制设计

为了降低学习效应与顺序效应影响，采用拉丁方（Latin Square）设计控制代码片段呈现顺序。

不同参与者接收到的测试样本顺序均不同，从而避免：

- 学习效应；
- 疲劳效应；
- 顺序偏差。

3.4.4 抗锚定效应机制

为了避免参与者直接依赖智能建议而产生锚定偏差（Anchoring Bias），系统设计了双阶段评审流程。

第一阶段：

1. 阅读代码；
2. 自主定位缺陷；
3. 填写评审意见；
4. 提交初始判断。

第二阶段：

1. 激活智能辅助；
2. 查看系统建议；
3. 修改或补充评审意见；
4. 提交最终结果。

系统界面如图所示：

实验平台界面示意图（后续替换为实际截图）

图 1: 双阶段评审流程

3.4.5 注意力检测机制

为了剔除低质量数据，实验过程中随机插入注意力检测题。

例如：

请在第 5 行代码处进行框选，并输入指定文本内容。

未通过检测的实验记录将被直接剔除。

3.5 实验异常与缓解方案

3.5.1 推理延迟问题

实验开展期间，Method-C 组出现推理延迟问题。

当代码长度超过 150 行时，CodeLlama-13B 推理时间显著增加：

$$MeanLatency = 8.2s \quad (9)$$

相比正常状态：

$$BaselineLatency = 3.2s \quad (10)$$

存在明显性能下降。

3.5.2 异步预推理机制

为缓解该问题，研究团队引入后台预推理流水线（Prefetching Pipeline）。

其工作流程如下：

1. 用户评审第 N 题；
2. 后台预加载第 N+1 题；
3. 提前执行 RAG 检索；
4. 完成 LLM 推理缓存；
5. 用户请求时直接返回结果。

优化后平均响应时间降至：

$$Latency < 0.2s \quad (11)$$

3.5.3 界面占位提示机制

针对偶发网络波动，系统增加占位提示组件：

“小助手正在思考中，请稍候……”

从而降低等待焦虑。

3.5.4 异常数据清洗

对于系统故障导致的异常记录，采用 IQR 方法识别异常值。
四分位距定义如下：

$$IQR = Q_3 - Q_1 \tag{12}$$

异常值判定条件：

$$x > Q_3 + 1.5IQR \tag{13}$$

或

$$x < Q_1 - 1.5IQR \tag{14}$$

共清洗 12 条异常记录。
随后采用组内中位数填补缺失数据，以保持统计稳定性。

4. 案例研究结果与实证分析

4.1 描述性统计结果

经过数据清洗后，共获得 1680 条有效分析单元记录。
表 5展示了不同方法在各项指标上的整体表现。

表 5: 核心指标描述性统计结果				
指标	Method-A	Method-B	Method-C	Control
Recall	0.62 ± 0.18	0.71 ± 0.15	0.83 ± 0.12	0.58 ± 0.21
Precision	0.78 ± 0.14	0.82 ± 0.11	0.89 ± 0.09	0.75 ± 0.16
F1-Score	0.69 ± 0.15	0.76 ± 0.12	0.86 ± 0.10	0.65 ± 0.17
审核时长（秒）	45.2 ± 9.3	39.8 ± 7.6	31.5 ± 6.2	52.7 ± 11.4
定位延迟（秒）	12.3 ± 3.1	10.1 ± 2.4	5.8 ± 1.7	15.4 ± 4.2
NASA-TLX	42.3 ± 8.1	38.6 ± 7.4	32.1 ± 6.8	48.9 ± 9.7
信任度	3.2 ± 0.9	3.8 ± 0.7	4.4 ± 0.6	N/A

4.2 RQ1: AI 生成代码是否增加人工评审负担？

为了验证 AI 生成代码是否会显著增加人工审阅者的认知负担，本研究首先在纯人工评审组（Control）内部进行纵向对比分析。

参与者分别审查：

1. 人类开发者编写代码；
2. 大模型生成代码。

采用配对样本 t 检验分析 NASA-TLX 认知负荷差异。

4.2.1 认知负荷分析

结果表明：

表 6: 不同代码来源下认知负荷比较

代码来源	NASA-TLX 均值	标准差
人工编写代码	36.4	8.2
AI 生成代码	48.9	9.7

配对样本检验结果：

$$t(7) = 4.82, \quad p < 0.01 \quad (15)$$

结果显示 AI 生成代码显著增加了评审认知负担。

4.2.2 审核时间分析

进一步分析发现：

表 7: 审核耗时比较

代码来源	平均耗时（秒）
人工代码	38.2
AI 代码	52.7

审核时间增长幅度达到：

$$\frac{52.7 - 38.2}{38.2} \times 100\% = 37.96\% \quad (16)$$

4.2.3 质性分析结果

采用扎根理论（Grounded Theory）对访谈文本进行三级编码分析。
最终归纳出两类核心困难模式。

(1) 伪完美结构带来的认知欺骗 72% 参与者提到：

AI 生成代码通常拥有规范命名、整齐缩进以及完整注释，因此容易产生“代码已经正确”的错觉。

这种现象被归纳为：

Cognitive Deception

即认知欺骗效应。

(2) 高隐蔽逻辑缺陷难以推演 参与者普遍反映：

- 并发死锁；
- 异常传播漏洞；
- 权限绕过；
- 类型转换错误；

等高层逻辑问题明显增多。

相比传统缺陷：

- 空指针；
- 拼写错误；
- 死循环；

更难通过人工快速发现。

因此，RQ1 得到验证：

AI 生成代码显著增加了人工审核的认知负担与审核耗时。

4.3 RQ2：智能评审支持工具的效果如何？

为了验证不同智能支持方法是否能够提高代码评审质量，本研究采用单因素方差分析 (One-Way ANOVA)。

4.3.1 前提检验

首先进行方差齐性检验。

Levene 检验

$$p = 0.214 > 0.05 \quad (17)$$

满足方差齐性要求。

同时各组数据通过正态性检验。

因此可以采用参数检验。

4.3.2 ANOVA 结果

以 F1-Score 作为因变量。

以评审方法作为自变量。

结果如下：

$$F(3, 92) = 12.67 \quad (18)$$

$$p < 0.001 \quad (19)$$

偏效应量：

$$\eta^2 = 0.29 \quad (20)$$

属于大效应量 (Large Effect)。

4.3.3 事后检验

采用 Bonferroni 校正。

结果如表 8所示。

表 8: Bonferroni 事后比较结果

比较组	均值差	95 %CI	p 值
Method-C vs Control	0.21	[0.14,0.28]	< 0.001
Method-B vs Control	0.11	[0.04,0.18]	0.003
Method-A vs Control	0.04	[-0.03,0.11]	0.382

结果表明：

- Method-C 效果最显著；
- Method-B 存在明显提升；
- Method-A 提升不显著。

因此拒绝原假设。

RQ2 得到支持。

4.4 RQ3：不同技术路径是否存在显著差异？

4.4.1 准确率比较

由于 Precision 指标未满足正态分布要求，因此采用 Kruskal-Wallis H 检验。

检验结果：

$$\chi^2(3) = 18.92 \quad (21)$$

$$p < 0.001 \quad (22)$$

效应量：

$$\varepsilon^2 = 0.17 \quad (23)$$

属于中等偏大效应。

4.4.2 Dunn 事后检验

结果如下：

表 9: Precision 两两比较

比较组	p 值	显著性
Method-C vs Method-A	< 0.001	显著
Method-C vs Method-B	0.012	显著
Method-B vs Method-A	0.094	不显著

4.4.3 效率比较

混合效应模型分析发现：

表 10: 审核效率比较

方法	平均耗时（秒）	相对降低
Control	52.7	—
Method-A	45.2	14.2%
Method-B	39.8	24.5%
Method-C	31.5	40.4%

4.4.4 主观体验比较

工具信任度结果如下：

表 11: 工具信任度比较	
方法	信任度均值
Method-A	3.2
Method-B	3.8
Method-C	4.4

访谈数据显示：

- 42% 参与者认为 Method-A 误报较多；
- 71% 参与者认为 Method-C 解释更清晰；
- 76% 参与者认为 Method-C 更符合真实评审流程。

因此 RQ3 得到支持。

4.5 RQ4：代码特征是否影响智能支持效果？

为了研究代码复杂度与长度的调节效应，本研究建立如下回归模型：

$$F1 = \beta_0 + \beta_1 Method + \beta_2 Complexity + \beta_3 Length + \beta_4 (Method \times Complexity) + \beta_5 (Method \times Length) + \epsilon \quad (24)$$

4.5.1 回归分析结果

表 12: 回归模型结果					
变量	β	SE	t	p	95
Method-C	0.24	0.05	4.80	< 0.001	[0.14,0.34]
Method-B	0.13	0.04	3.25	0.002	[0.05,0.21]
Complexity	-0.09	0.02	-4.50	< 0.001	[-0.13,-0.05]
Method-C×Complexity	0.14	0.04	3.50	0.003	[0.06,0.22]
Method-C×Length	0.09	0.03	3.00	0.021	[0.03,0.15]
Method-A×Complexity	-0.07	0.03	-2.33	0.048	[-0.13,-0.01]

4.5.2 调节效应解释

研究发现：

- 代码复杂度越高，人类评审质量越低；

- Method-A 在高复杂度场景下进一步退化；
- Method-C 在高复杂度场景下优势进一步增强。

这表明 LLM 智能体具有典型的：

Difficult-Task Amplifier

即“高难度任务放大器”效应。
随着任务难度提升，其边际收益持续增加。

4.6 认知卸载—决策增强协同模型

结合行为日志与访谈结果，本研究提出智能代码评审的人机协同机制模型。

4.6.1 路径 A：认知卸载

智能体提前完成：

- 控制流分析；
- 异常传播分析；
- 安全漏洞定位；
- 缺陷原因解释。

从而减少人工 Dry-Run 推演过程。

76

智能体显著减少了我在脑海中模拟代码执行的时间。

4.6.2 路径 B：决策增强

Method-C 不仅指出问题。

同时生成：

- 修复建议；
- 重构方案；
- 可直接应用的代码片段。

使定位延迟从：

$$15.4s \rightarrow 5.8s \tag{25}$$

降低：

$$62.3\% \tag{26}$$

4.6.3 Human-in-the-Loop 特征

值得注意的是：
即使在 Method-C 条件下，
参与者仍会修改：

23% (27)

的 AI 建议。
主要包括：

- 变量命名；
- 异常处理级别；
- 项目业务逻辑适配；
- 安全策略调整。

这说明智能支持并未削弱人类决策能力。
相反，人类依然是最终质量控制者。
因此提出：

认知卸载 (Cognitive Offloading) → 决策增强 (Decision Enhancement)

双路径协同机制。

5. 案例效度威胁与缓解策略分析

根据实证软件工程 (Empirical Software Engineering) 关于案例研究设计标准规范，本研究从内部效度、外部效度、构念效度以及统计结论效度四个方面系统分析潜在威胁，并针对性采取缓解措施。

5.1 内部效度 (Internal Validity)

内部效度关注实验结果是否真正由研究中的自变量所导致，而非受到其他混杂因素影响。

本研究主要面临以下潜在威胁：

1. 不同参与者能力差异导致的基线偏差；
2. 长时间实验引发的学习效应与疲劳效应；
3. 智能评审工具响应延迟产生额外干扰；

4. 个体对 AI 工具熟悉程度差异。

为降低上述影响，本研究采用如下控制策略：

- 预实验筛选参与者能力；
- 分层随机化分组；
- 拉丁方实验设计；
- 统一培训流程；
- 推理延迟统一优化至 200ms 以内。

5.2 外部效度 (External Validity)

外部效度关注研究结论能否推广到真实工业场景。

本研究存在如下限制：

1. 参与者主要来自高校环境；
2. 缺少资深工业架构师参与；
3. 实验代码长度控制在 200 行以内；
4. 未覆盖超大型系统场景。

为提升外部效度，本研究采取以下措施：

- 引入真实 GitHub 项目代码；
- 使用真实缺陷修复案例；
- 覆盖不同复杂度等级；
- 涵盖 Java 与 Python 两类主流语言。

未来计划在企业级软件研发团队中开展多中心验证研究。

5.3 构念效度 (Construct Validity)

构念效度关注实验指标是否真正测量了研究目标。

代码评审质量本质上是复杂且多维的概念。

若仅依赖单一指标进行评价，可能导致结论失真。

因此本研究采用多维评估体系：

表 13: 构念效度保障措施

评估维度	对应指标
质量	Recall、Precision、F1-Score
效率	审核时长、定位延迟
认知负荷	NASA-TLX
用户体验	可理解性评分
信任关系	工具信任度量表

此外，为降低霍桑效应（Hawthorne Effect）：

- 未向参与者公开具体研究假设；
- 强调自然评审行为；
- 设置练习任务降低实验紧张感。

5.4 统计结论效度（Conclusion Validity）

统计结论效度关注数据分析是否科学合理。

潜在威胁主要包括：

1. 多重比较导致第一类错误增加；
2. 偏态分布数据错误使用参数检验；
3. 样本量不足导致统计功效下降。

对应缓解措施如下：

- 采用 Bonferroni 校正；
- 采用 Dunn 事后检验；
- 正态性与方差齐性前置检验；
- 非参数条件下使用 Kruskal-Wallis 检验；
- 报告效应量指标。

5.5 效度威胁汇总

表 14: 案例研究效度威胁与缓解措施总结

效度类别	潜在威胁	缓解措施
内部效度	能力差异、疲劳效应、延迟影响	分层随机化、拉丁方设计、统一环境
外部效度	高校样本、代码规模有限	真实开源项目、多样化代码场景
构念效度	单指标无法完整衡量评审质量	构建多维评价指标体系
统计结论效度	多重比较与统计假阳性	Bonferroni 校正与非参数检验

6. 研究启示

6.1 对智能评审工具设计者的启示

研究表明，直接展示 AI 建议可能导致用户产生锚定效应（Anchoring Bias）。因此未来代码评审工具应采用：

独立评审 → AI 辅助评审

的双阶段设计模式。
具体建议包括：

- 1. 强制用户先完成初始判断；
- 2. 再解锁智能建议；
- 3. 保留用户修改轨迹；
- 4. 支持建议解释与证据展示。

这种设计能够有效促进主动思考。

6.2 对软件工程团队管理者的启示

传统 CI/CD 流水线往往依赖静态分析工具。
然而本研究发现：
Method-A 虽然能够发现部分规范性问题，但对复杂逻辑缺陷的帮助有限。
因此建议采用分层门禁策略：

表 15: 智能代码评审分层策略

代码类型	推荐方法
格式规范检查	Method-A
中等复杂业务逻辑	Method-B
高复杂核心模块	Method-C
安全关键代码	Method-C
并发系统模块	Method-C

这种策略能够兼顾：

- 成本；
- 响应速度；
- 审查质量。

6.3 对软件工程研究者的启示

研究进一步证明：

AI 并不会完全取代代码评审人员。

相反：

Human-in-the-Loop

依然是当前最优协同模式。

未来研究应重点关注：

- AI 建议可解释性；
- 人类修正行为建模；
- 协同决策机制优化；
- 长期信任关系演化。

7. 研究局限性

尽管本研究在实验设计与执行过程中尽可能保证严谨性，但仍然存在以下局限。

7.1 模型迭代带来的技术时效性问题

Method-C 基于 CodeLlama-13B 构建。

随着大语言模型快速发展：

- GPT 系列；
- Claude 系列；
- Gemini 系列；
- DeepSeek 系列；

均可能在未来表现出不同能力水平。

因此：

本研究所得绝对性能指标可能随底座模型变化而变化。

但不同技术路径之间呈现的整体趋势仍具有参考价值。

7.2 实验规模限制

受实验时间与资源约束：

- 样本数量为 212 个代码单元；
- 参与者数量为 24 人；
- 代码长度不超过 200 行。

因此对于超大型项目：

- 微服务架构；
- 分布式系统；
- 云原生平台；

的适用性仍需进一步验证。

7.3 组织环境因素缺失

实验室环境难以完全复现真实企业环境。

例如：

- 上线压力；
- 业务风险；
- 团队文化；
- 权威影响；
- 时间截止压力。

这些因素均可能影响代码评审行为。

8. 未来工作展望

基于本研究发现，未来研究可从以下方向进一步推进。

8.1 多模态代码评审智能体

当前系统主要依赖静态代码文本。

未来可融合：

- 控制流图（CFG）；
- 数据流图（DFG）；
- 测试覆盖率；
- 系统日志；
- 历史 Pull Request 记录；
- 架构设计文档。

实现更加全面的上下文理解能力。

8.2 工业级长期纵向研究

未来计划将智能评审系统部署至企业研发环境。

开展：

- 6 个月跟踪研究；
- 12 个月纵向研究；
- 多中心协同实验。

重点观察：

- 技能迁移；
- 长期依赖；
- 团队协作变化；
- 信任动态演化。

8.3 透明化与可信 AI 机制

未来需要进一步研究：

- AI 建议置信度展示；
- 推理过程可解释化；
- 不确定性可视化；
- 责任归因机制。

从而建立更加可靠的人机协同关系。

9. 结论

随着生成式人工智能技术逐渐融入软件开发流程，代码评审活动正在发生深刻变革。

本研究围绕“面向 AI 生成代码的人工审核与智能支持现状”开展了系统性的经验软件工程案例研究。

研究结果表明：

1. AI 生成代码显著增加了人工评审负担与认知成本；
2. 智能评审支持技术能够有效提高评审质量；
3. LLM 评审智能体在质量、效率与用户体验方面均表现最佳；
4. 高复杂度代码场景下，LLM 辅助效果更加显著；
5. 人机协同仍是当前最优代码评审范式。

进一步地，本研究提出了“认知卸载—决策增强（Cognitive Offloading – Decision Enhancement）”双路径协同模型，用于解释智能支持技术提升代码评审绩效的底层机制。

总体而言，本研究为软件工程 3.0 时代智能代码评审系统的设计、部署与优化提供了重要的实证依据，同时也为未来人机协同软件开发模式研究奠定了基础。